P&ID Digitizer with Design-Rule Checker: Technical Document

Nov 5, 2026 · @Sanika Mhatre

Overview

The system turns a scanned piping and instrumentation diagram (P&ID) into a verified graph, a bill of materials and a list of design-rule violations, each pointing at the exact place on the drawing. It was built for the Meta Global AI Developer Hackathon (manufacturing and energy) and runs on Meta's SAM 3.1 and Muse models through the Meta Model API.

The problem. Process plants keep decades of drawings as images. Which pump feeds which vessel, and which valves isolate what, is locked in pixels. That blocks plant modifications, safety reviews and digital twins, and checking a drawing against design rules is manual work.

Input and output

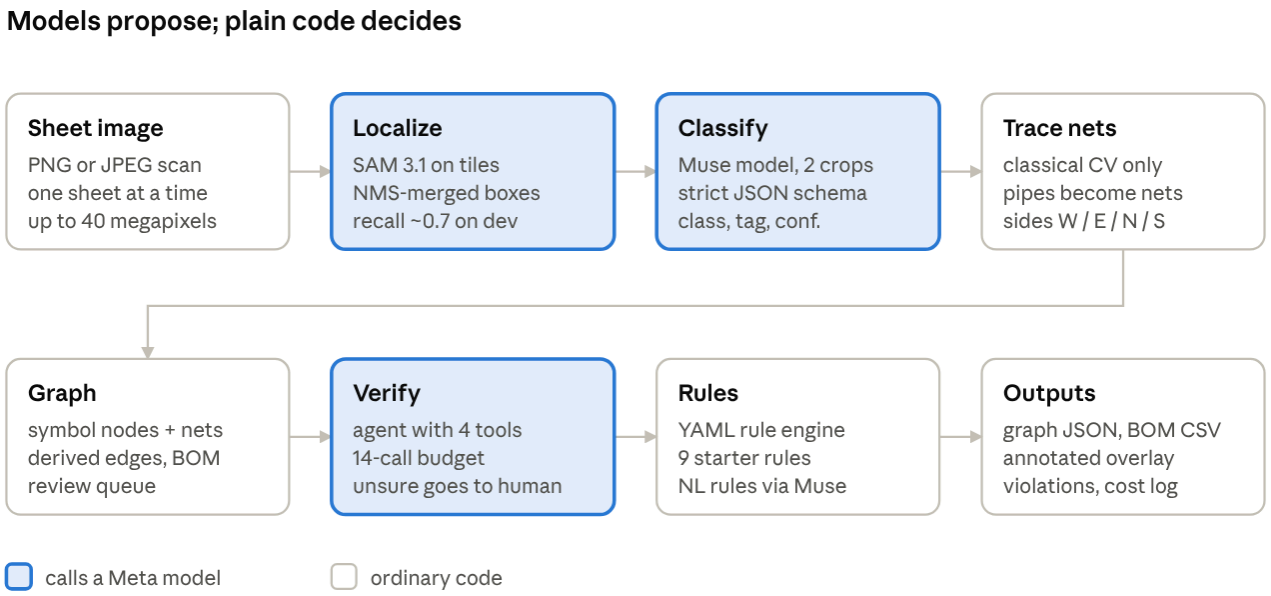
- Input: one sheet image (PNG or JPEG).
- Graph JSON: symbols with class, tag and confidence, plus the pipes and signal lines (nets) that connect them.
- Bill of materials as CSV.
- Annotated overlay of the original drawing.
- Design-rule violations, each tied to a symbol or net on the drawing.
- Warnings, a human review queue, and a cost ledger for the run.

Design principle: models propose, deterministic code decides. The Muse model proposes classes, tags and links with confidences. A verification agent re-examines the uncertain ones using tools. The rule engine that raises violations is ordinary code over the graph, so every finding is reproducible, testable and auditable. A language model can author a rule, but it can never decide whether a rule is violated.

Scope of the evidence. Every measurement in this document comes from generated sheets with planted errors. No results on real scanned drawings exist yet.

System architecture

A sheet passes through eight stages in order, and only three of them call a model. Everything that decides a result, the tracing, the graph and the rules, is ordinary code.



pipeline · 8 stages, 3 call a Meta model

Localize finds symbol boxes, Classify names each box and reads its tag, and Trace turns the remaining pipe pixels into nets. The Graph then goes to Verify, where an agent re-examines only the uncertain items, and the Rules stage raises violations. Outputs are the graph JSON, BOM, overlay, violations and a cost log.

Models and their jobs

Model	Where	Job
SAM 3.1 (`sam-3.1` , Responses API)	Localize	finds symbol bounding boxes from short text prompts, tile by tile
Muse Spark 1.3 (`muse-spark-1.3` , chat completions), vision	Classify	class, tag and confidences from two crops, as strict JSON
Muse Spark 1.3, tool calling	Verify	the verification agent
Muse Spark 1.3, structured output	Rules	translates a plain-English rule into the YAML schema
Muse Spark 1.3, vision (optional)	Localize	alternative localizer, or merged with SAM, for comparison

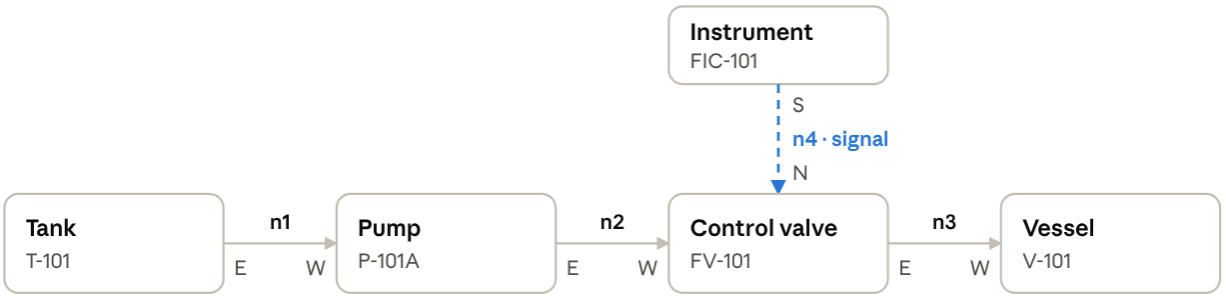
Both model families are served by the Meta Model API with one key. The localizer is a plug-in (`PID_LOCALIZER=sam` , `muse` or `both`), so the approaches can be compared on the same sheets.

Data model

A sheet is a graph with two kinds of element: symbols as nodes, and pipes or signal lines as nets. A net records which symbols it touches and on which side, so flow direction and junctions are kept without guessing. The schema is a set of pydantic models (`schema.py`), validated on load and serialized to JSON.

A net records which side of each symbol it touches

Example sheet with illustrative tags



Flow reads west to east: a symbol's west side is its inlet and its east side its outlet.
n2 holds two attachments, (P-101A, E) and (FV-101, W). A tee would add a third to the same net.

process net signal net

graph example · 4 symbols, 3 process nets, 1 signal net

Net n2 joins the pump's east side to the control valve's west side. Net n4 is a dashed signal net from the instrument to the valve's north side.

Element	Fields	Notes
Node	id, class, tag, bbox, confidence, tag_confidence, source, needs_review, notes	bbox in sheet pixels (x0, y0, x1, y1)
Attachment	node, side	side is W, E, N or S
Net	id, type, attachments, bbox, length_px, confidence	type is process or signal
Graph	sheet_id, width, height, nodes, nets, review_queue	derived: edges, connected pairs, BOM

The eight symbol classes are tank, vessel, pump, valve, check_valve, control_valve, instrument and relief_valve.

Why nets, not edges. A tee that feeds two valves is one net with three attachments, not a clique of three edges. Storing the net keeps junctions honest, and the node-to-node edge list is derived from it when a rule or an export needs one.

Flow direction from the side of attachment. By convention a symbol's inlet is its west side and its outlet its east side. That is what lets a rule say "downstream of the pump". Process nets are drawn as solid lines and signal nets as dashed ones, so a control loop is distinguishable from piping.

Review queue. Items the system could not settle, whether by confidence or by agent budget, are written into the graph's review queue with the evidence, rather than guessed.

Localization

Localization finds a bounding box for every symbol; everything downstream depends on it, so its recall is the ceiling for the whole pipeline. The default localizer is SAM 3.1 called through the Meta Model API. Two alternatives share the same interface and can be swapped with one setting (`PID_LOCALIZER`).

SAM 3.1 (default). The sheet is cut into overlapping tiles (1280 px, 128 px overlap). Each tile is sent once per text prompt as a `POST /v1/responses` request with model `sam-3.1` , an `input_text` phrase and an `input_image` data URL. The reply is text: for each match a box token such as `<|box;x1=..;y1=..;x2=..;y2=..;w=..;h=..;c=..|>` followed by a mask. Only the boxes are used. The parser reads the tokens, treats x2 and y2 as inclusive pixel coordinates, scales by the reported w and h, drops matches under confidence 0.3, and offsets the box by its tile origin. Boxes from all tiles and prompts are merged with non-max suppression (IoU 0.5, plus suppression of boxes mostly contained in a kept one) and filtered by size (at least 14 px a side, at most 30% of the sheet). SAM is billed per image at $2.50 per 1,000, which the cost ledger records per call.

Prompt phrases were chosen by measurement. SAM takes one short noun phrase per request. On a generated dev sheet (seed 0, 16 symbols), a sweep of 19 phrases showed that domain words ("pump", "valve", "tank", "vessel", "instrument") returned no boxes at all, while generic ones worked: "symbol" matched 9 of 16 ground-truth symbols, "rectangle" matched the tanks and "circle" matched a pump and an instrument. The defaults are therefore `symbol` , `rectangle` and `circle` , overridable with `SAM_PROMPTS` . The class is never taken from the phrase: the Muse classifier reads each box.

Muse localizer (alternative). The Muse model is asked for tight boxes tile by tile (768 px tiles, 96 px overlap) with the class legend, returning JSON. Answers given in normalized coordinates are scaled up, and unreadable tiles are skipped. It has no fixed per-image price and was weaker on one of the two sheets.

Merged localizer. `both` takes the union of the two and applies non-max suppression.

Measured on two generated dev sheets (seeds 0 and 1), IoU 0.5

Localizer	Recall	Precision	Localization cost per sheet
SAM, prompts symbol / rectangle / circle	0.75, 0.71	0.80, 0.71	about $0.01
Muse	0.75, 0.36	not recorded	about $0.03
SAM and Muse merged	0.88, 0.86	0.67, 0.48	about $0.035

Merging finds more symbols but adds false boxes. The Evaluation section shows how that plays out end to end.

Classification and tag reading

Every candidate box is sent to the Muse model (`muse-spark-1.3`, Standard mode) with two crops: a tight crop of the symbol and a wider crop around it where its tag usually sits, below or to the right. The prompt carries a legend that describes each of the eight classes in words, plus a `not_a_symbol` class for text, pipe fragments and noise.

Strict structured output. The reply must match a JSON schema with five required fields and no extras: `class` (an enum), `class_confidence`, `tag` (string or null), `tag_confidence` and `evidence` (one short sentence). The system prompt forbids inventing a tag: when none is visible next to the symbol the model returns null with low confidence. A class outside the enum is coerced to `not_a_symbol`, and confidences are clamped to 0 to 1.

Failure handling. Authentication, network and quota errors are raised and stop the run, so a bad key can never produce an empty graph that looks like a clean sheet. Only malformed model output is absorbed, and then for one symbol: it becomes `not_a_symbol` with confidence 0 and the failure noted in its evidence.

Reasoning effort. Routine crops use low effort. Disputed items are asked again at medium or high effort inside the verification loop. The model is a reasoning model, so a small token budget can be used up by thinking and return an empty answer. The client handles that case (see the client section).

Throughput and cost control. Candidates are classified in a thread pool (8 workers), every request is cached on disk, and a hard cap (`max_candidates`, default 300) limits how many model calls one sheet can trigger. If the cap bites, the highest-scoring candidates

are kept and a warning says so.

Legend. The default legend matches the generated sheets. For a real drawing set, `parse_legend` asks the model to describe a legend image class by class and falls back to the default if that fails.

Net tracing

Connectivity is computed with classical image processing, not by a model. Once the symbol boxes are known, the pipe pixels are what remains, and their connected components are the nets (`trace.py`, using numpy, scipy and scikit-image).

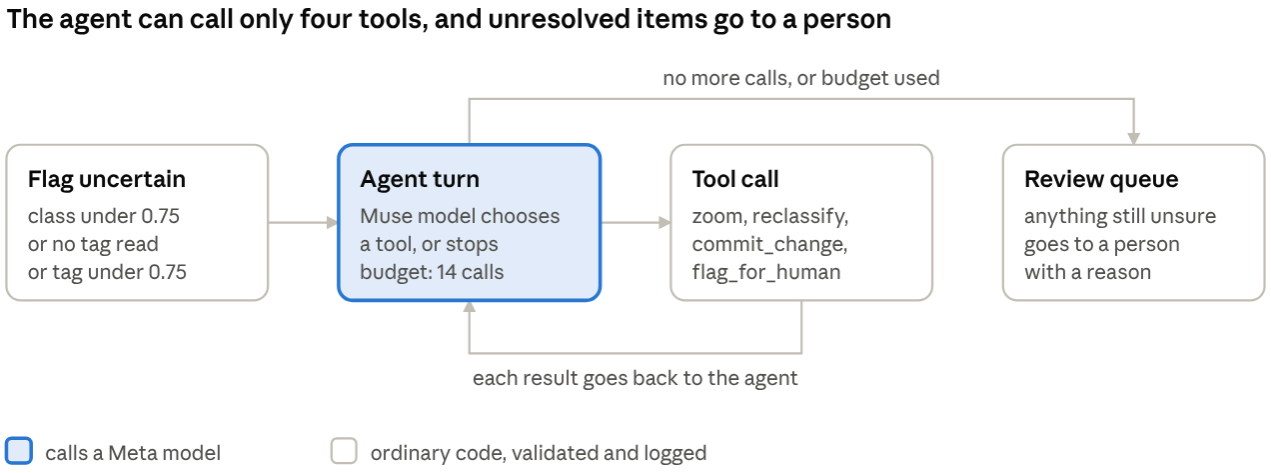
1. **Binarize** the sheet at an ink threshold of 170.
2. **Mask every symbol box**, padded by 6 px, so only pipes and text remain.
3. **Bridge dashes** with a directional closing of up to 8 px, so a dashed signal line becomes one connected shape.
4. **Label connected components.** Each component with a skeleton at least 10 px long is a candidate net.
5. **Reject text.** Tag labels also survive masking. Small strips (at most 30 by 160 px) are treated as text when their skeleton is long relative to their size (more than 1.2 times width plus height, which a straight pipe never is) or their ink ratio is below 0.5.
6. **Attach to symbols.** A net attaches to every symbol whose padded box it touches within 6 px. The side (W, E, N or S) comes from where the net's pixels sit relative to the symbol's centre.
7. **Classify the net type.** A component whose ink fills less than 80% of its bridged area is a dashed line and becomes a `signal` net; the rest are `process` nets.

Because attachment depends on the boxes, a symbol the localizer missed breaks every net that passed through it. That is the main reason connectivity scores trail symbol scores in the evaluation.

Known limits. Pixel connectivity cannot tell a line crossing from a junction, so both merge. Tag text that touches a pipe is read as part of the pipe. Both are listed under limitations.

Verification agent

The agent re-examines uncertain symbols with tools instead of guessing, and it can never edit the graph directly. Its only actions are four whitelisted tools whose arguments are validated; every call is appended to an audit log.



agent loop · 4 tools, 14-call budget

Each turn the agent picks a tool or stops. Every call is logged, and whatever stays unresolved is queued for a person.

What gets verified. A symbol is sent to the agent when its class confidence is below 0.75, when no tag was read, or when its tag confidence is below 0.75. The agent receives that list with the reasons.

Tool	What it does
`zoom`	returns a magnified crop around a symbol, attached in the next message
`reclassify`	asks the classifier again at a higher reasoning effort (high by default)
`commit_change`	writes a corrected field with the evidence for it
`flag_for_human`	marks the symbol as needing review and adds it to the review queue with a reason

Budget and loop. The agent runs at medium reasoning effort with a fixed budget of 14 tool calls per sheet. Each turn it either calls tools or stops. Calls past the budget return an error instead of running.

No silent guesses. When the loop ends, any flagged symbol that is still low-confidence or still untagged, and that the agent did not touch, is added to the review queue with the reason "unresolved after agent budget". Flagging for a human is a normal, first-class outcome.

Without a model. `flag_only` applies the same uncertainty test and queues the items for a person without calling any model, so the system degrades to a review list rather than failing.

The agent can be switched off (`--no-agent` in the evaluation, a checkbox in the UI), which lets its contribution be measured.

Rule engine and natural-language rules

Violations come from a deterministic engine over the graph. Rules are data (YAML) validated by pydantic models that forbid unknown fields, so a malformed rule is rejected rather than half-applied. No model call is involved in deciding whether a rule is violated.

Rule format. A rule has an id, a severity (error, warning or info), a description, an `applies_to` matcher and exactly one `require` condition:

```
- id: R4-PUMP-CHECK-VALVE
  severity: warning
  description: Pump discharge needs a check valve within 2 hops downstream
  applies_to: {class: pump}
  require:
    path: {direction: downstream, edge_type: process, max_hops: 2, reaches:
{class: check_valve}}
```

- Matchers select by `class`, `class_in`, `has: tag`, `missing`, or `kind: net`.
- Requirements are `path` (direction, edge type, hop count, target class), `unique: tag`, `has_attr: tag`, `connected: true`, and `min_symbols: N` for nets.
- Direction-aware paths use the side-of-attachment convention: downstream follows a symbol's east side.

Starter pack (nine rules). The pack is an illustrative example, not a statement of safety compliance; real plants apply their own standards.

Rule	Severity	Check
R1-UNIQUE-TAG	error	tags are unique on a sheet
R2-TAG-PRESENT	warning	every symbol carries a tag
R3-DANGLING-PIPE	error	a pipe attaches to at least two symbols
R4-PUMP-CHECK-VALVE	warning	check valve within 2 hops downstream of a pump
R5A-PUMP-SUCTION-ISOLATION	warning	isolation valve directly upstream of a pump
R5B-PUMP-DISCHARGE-ISOLATION	warning	isolation valve within 2 hops downstream of a pump

Rule	Severity	Check
R6-CONTROL-VALVE-LOOP	error	a control valve is linked to an instrument by a signal line
R7-VESSEL-RELIEF	warning	a pressure vessel has a relief valve connected
R8-ISOLATED-SYMBOL	error	a symbol is connected to something

Natural-language rules. A person can type a rule such as "every pump must have an isolation valve on both sides". The Muse model translates it into the YAML schema, with two worked examples in the prompt and the allowed classes listed. The output is then validated against the rule schema. Unknown classes and malformed conditions are rejected and sent back for one repair attempt. A valid rule is previewed on reference sheets and applied only after the person approves it. The model therefore authors rules; the engine, which is plain code, still decides every violation.

Model client and cost control

All model calls go through one small client (`models/llm.py`, built on httpx) so that retries, caching and cost accounting behave the same everywhere. It speaks the OpenAI-style `/chat/completions` protocol for the Muse model and can post to other endpoints on the same provider (`/responses` for SAM). Settings come from environment variables, which a small loader reads from a local `.env` file.

Setting	Purpose
`LLM_API_KEY`, `LLM_BASE_URL`, `LLM_MODEL`	the Muse model endpoint (Meta Model API: `https://api.meta.ai/v1`, `muse-spark-1.3`)
`PID_LOCALIZER`	`sam` (default), `muse` or `both`
`SAM_PROMPTS`, `SAM_API_KEY`, `SAM_BASE_URL`	SAM phrases and endpoint; the key and URL default to the Muse ones
`LLM_PRICE_IN`, `LLM_PRICE_OUT`	USD per million tokens, used for the cost estimate

Reliability. Network errors and HTTP 429, 500, 502, 503 and 504 are retried up to four times with exponential backoff capped at 20 seconds. A 400 that complains about `reasoning_effort` is retried once without that parameter. Any other error raises an `LLMError` carrying the status and the start of the body, so a wrong key or model id fails loudly.

Reasoning models and token budgets. The Muse model thinks before answering and spends `max_tokens` doing so. The first live smoke test returned empty text on three checks that used budgets of 20 to 100 tokens. The client now detects an empty reply with finish reason `length` and retries once with four times the budget, up to 8,000 tokens. The retry's cost is recorded too.

Cache. Each request is stored on disk under a SHA-256 hash of its path and payload, so repeating a run during development costs nothing. Results measured on cached calls are labelled as cached in the ledger.

Ledger. Every call is recorded with its purpose (classify, agent, localize and so on), latency, token counts, estimated cost and whether it was cached. Totals and per-purpose breakdowns feed the UI's Performance tab and the cost-per-sheet figure in the evaluation. The ledger is thread-safe because classification runs in a pool.

How to read the cost numbers. Token cost is computed from the configured prices, whose defaults ($1.25 in and $4.25 out per million tokens) are placeholders rather than a published Meta rate. SAM is the exception: it uses its listed price of $2.50 per 1,000 images. Cost per sheet in this document is therefore an estimate, to be recomputed with the provider's real prices.

Evaluation

On six generated dev sheets the default setup reads tags perfectly and finds 4 of 5 planted errors, but connectivity (0.46) and violation agreement (0.23) are weak, and the cause is localizer recall. All numbers are from generated sheets; none come from real scanned drawings.

Test data. `pid.synth` draws original P&ID sheets with the eight symbol classes, pipes, signal lines and tags, and can plant seven kinds of design error: missing check valve, missing isolation valve, missing tag, duplicate tag, dangling pipe, missing signal line and missing relief valve. Because the errors are planted, the expected violations are known by construction. Optional scan noise degrades the image. Seeds below 1000 are development sheets and seeds from 1000 up are held out and not used for tuning. The evaluation plans sheets so that every fourth one is clean and the others cycle through the error kinds.

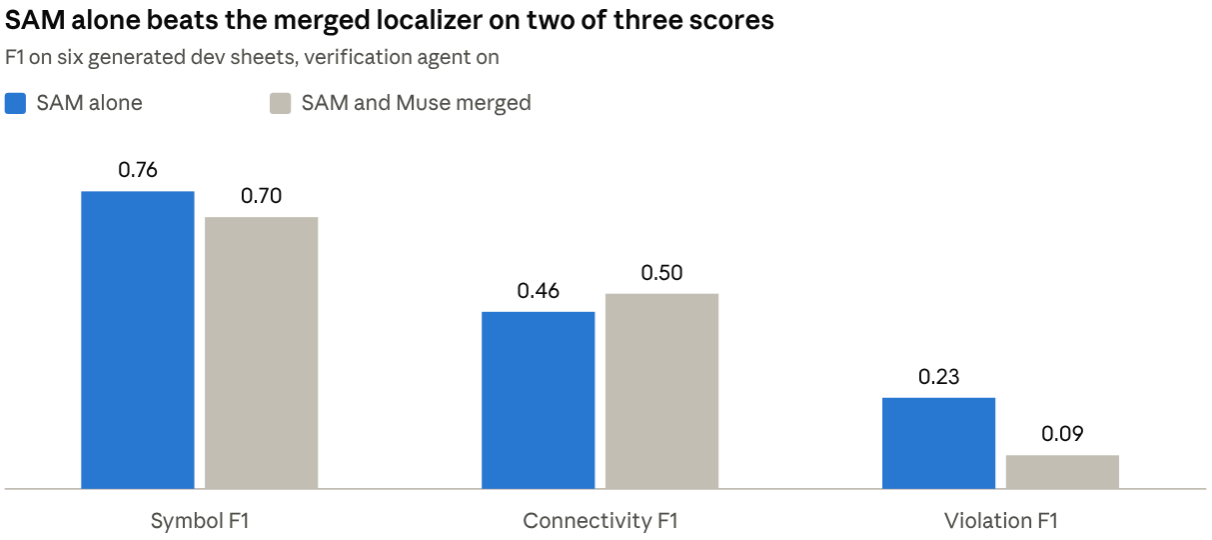
Metrics (symbols are matched to ground truth by IoU 0.5)

- Symbol localization and class-aware symbol F1.
- Tag read accuracy: among matched symbols that have a ground-truth tag, the share read exactly.
- Connectivity F1 over typed symbol pairs (process or signal).

- Violation F1: violations raised on the predicted graph against those the engine raises on the ground-truth graph.
- Planted-error recall, seconds per sheet, estimated cost per sheet and items sent to review.

The `simulated` mode replaces the model stages with ground-truth lookups. It exercises tracing, rules and metrics only, and its perfect scores are not model accuracy.

Results: dev seeds 0 to 5 (six sheets), live Muse Spark 1.3 and SAM 3.1, verification agent on



evaluation run · dev seeds 0 to 5, generated sheets

Localizer	Tag read acc.	Planted errors found	Seconds per sheet	Cost per sheet
SAM alone (default)	1.00	4 of 5	27.8	$0.056
SAM and Muse merged	1.00	5 of 5	86.6	$0.070

Reading the results

- The Muse model reads tags and classifies reliably. The weak numbers trace back to symbols the localizer misses: with SAM alone, symbol recall is 0.69 to 0.73 on every sheet, so three to four symbols per sheet are lost and the pipes around them no longer connect.
- Merging the localizers raised recall in the smoke test (0.86 to 0.88) but added false boxes, which lowered symbol and violation F1 and tripled the time per sheet. It found one more planted error. With six sheets that difference is a single error.

- The planted error SAM-only missed was a missing tag on a sheet where recall was 0.71.
- Cost is a few cents per sheet. It uses the configured token prices (see the client section), so treat it as an estimate.

Not yet measured. The verification agent's contribution (`--no-agent` ablation), scan-noise sheets, the held-out test seeds, and any real scanned drawing.

Security, limitations and reproduction

Security and privacy

- **Prompt injection.** Drawings can contain arbitrary text. Model outputs are schema-constrained, the agent's tools are a closed whitelist with validated arguments, and nothing a model returns is executed or used as a rule verdict. The worst a hostile tag can do is produce a wrong tag value, which goes to the review queue when confidence is low.
- **Confidential drawings.** Use the Standard model, not the `-contributor` variant, which lets Meta train on inputs. Responses are cached on disk (`PID_CACHE_DIR`); deleting `.cache/` purges them. Request bodies containing images are not written to logs.
- **API server.** Upload size limit (15 MB) and pixel limit (40 million), optional bearer-token authentication with a constant-time comparison, request ids, no upstream error details returned to clients, and a non-root container.
- **Secrets.** Read from the environment or a git-ignored `.env` ; never committed.

Limitations

- Localizer recall is the main accuracy limit (about 0.7 with SAM alone on generated sheets).
- Line crossings and junctions both merge under pixel connectivity. Tag text that touches a pipe is read as part of the pipe.
- Flow direction assumes west to east. Other layouts need the attachment convention generalized.
- One sheet at a time: off-page connectors and cross-sheet consistency are not handled. Pipe sizes, line numbers and specifications are not read.
- Generated sheets are simpler than real scans; real accuracy is unknown.
- The Muse model used is `muse-spark-1.3` from the Meta Model API. Muse Spark Open is distributed as downloadable weights and served by hosted providers; the client is model-agnostic through `LLM_BASE_URL` and `LLM_MODEL` , but that configuration has not been run.

- The starter rules are illustrative, not a compliance standard.

Repository layout

```
src/pid/
  schema.py          graph schema
  rules/             engine.py, starter_pack.yaml, nl_compiler.py
  segment/           meta_sam.py, muse_localizer.py, combined.py, base.py
(tiling, NMS)
  classify.py        Muse classifier and legend parsing
  trace.py           classical net tracing
  agent.py           verification agent
  pipeline.py        orchestration, warnings, cost accounting
  models/llm.py      client: retries, cache, ledger
  synth/generator.py generated sheets with planted errors
  eval/              metrics.py, run_eval.py
  app/               api.py (FastAPI), ui.py (Streamlit)
  smoke.py           live checks for the endpoint and the localizer
tests/               automated tests, no API key needed
```

Reproduce

1. Create an environment and install: `conda create -n pid python=3.12`, `conda activate pid`, `pip install -e ".[dev]"`, then `pytest -q`.
2. Create an API key at dev.meta.ai, copy `.env.example` to `.env`, and set `LLM_API_KEY`, `LLM_BASE_URL=https://api.meta.ai/v1` and `LLM_MODEL=muse-spark-1.3`.
3. Check the connection with `python -m pid.smoke llm` and `python -m pid.smoke localize`.
4. Run the UI with `streamlit run src/pid/app/ui.py`, choose Live models, and upload `demo/planted_errors.png`.
5. Measure with `python -m pid.eval.run_eval --mode live --n 6`, adding `--localizer both` or `--no-agent` for ablations.